HPL on the NUS SoC Compute Cluster: how I got 8.55 TFLOPS on two nodes

Kent Ridge HPC try-out, Option 1 (software benchmarking). Prepared by Ethan. DRAFT.

Result. The best full-size run, run from the unpacked submission archive (`make`, then `make run`), reached **8549.9 GFLOPS** (N = 231,552, NB = 384, 968 s, residual check PASSED) on 2 xcni nodes. That is 58.8% of an *assumed* Rpeak of 14,540.8 GFLOPS. Three identical full-size runs scored 7701.6, 8309.3 and 8549.9 GFLOPS, an 11% spread on a shared cluster, so the number a reader sees will land somewhere in that range.

1. Setup

- **Hardware (xcni class).** 2 x AMD EPYC 9355 (Zen 5), 64 cores / 128 threads and 250 GB per node, AVX-512, 10 GbE (TCP only, no InfiniBand). The login nodes are no-AVX virtual machines, so nothing is tuned there and all builds run on a compute node via `srun`.
- **Software.** HPL 2.3, AOCL-BLIS 5.0 built as *amdzen* (one binary that picks its kernel at run time; it chose *zen5*), and a private Open MPI 4.1.6 with bundled PMIx/hwloc/libevent. One build serves both node types, so there is no `-march=native`.
- **Layout.** Hybrid MPI + OpenMP: 16 ranks per node x 4 threads, bound with `--map-by slot:PE=4 --bind-to core`, OMP_PLACES=cores, OMP_PROC_BIND=close. Each rank is given whole cores (hyper-thread siblings included) or Open MPI's core binding fails.
- **Rpeak (assumed).** 64 cores x 3.55 GHz x 32 DP FLOP/cycle = 7,270.4 GFLOPS per node, 14,540.8 for two. The 3.55 GHz base clock comes from the vendor spec and could not be read on the node (it exposes only a 4.4 GHz boost), and 32 FLOP/cycle assumes full-rate AVX-512. Percentages are therefore estimates.

2. Constraints I ran into

- **Two nodes, not four.** Slurm's QOS caps a user at 256 CPUs (`sacctmgr show qos`). One xcni node is 128 CPUs, so a 3- or 4-node job sits in the queue with `QOSMaxCpuPerUserLimit` forever. I confirmed it with test jobs: 2 nodes (256 CPUs) ran, 3 (384) and 4 (512) stayed pending. Running 4 nodes at half the cores would use the same 256 CPUs but add network traffic, so I used 2 full nodes, which is also the task's minimum.
- **System Open MPI is unusable here** (PMIx segfaults, lost daemons), hence the private build and a launch with its own `mpirun` inside `sbatch`.
- **The first 2-node job hung.** The ranks sat at 99.9% CPU with about 10 MB resident each (no matrix allocated) and the log said `connect() to 192.168.58.21 failed`. Each node has two NICs and Open MPI picked the one on a subnet that is not routable between nodes. Pinning `btl_tcp_if_include` and `oob_tcp_if_include` to the working 192.168.48.0/21 fixed it. I used the subnet rather than an interface name so it holds on any node.
- Smaller things: `--exclusive` is refused and the default memory is 3 GB per CPU, so whole-node jobs need `--mem=0`; the xcng nodes are ARM, so every job is pinned with `--constraint=xcni`; Open MPI compiles on node-local /tmp because the NFS home is too slow.

3. Tuning journey

One variable at a time, keeping a change only if it beat the run-to-run noise (about 1.5% at small N). I tuned on a single node first (N = 46,080), then on two nodes (N = 65,280 to 67,200) and confirmed on the full size.

Single node: block size, grid, ranks x threads (GFLOPS)

NB (64 ranks, 8x8)	192	256	320	384	512
GFLOPS	2358	2684	3009	3173	2774

Ranks x threads (NB 384)	64x1	32x2	16x4	8x8	2x32
GFLOPS	3282	3476	3703	3340	2241

Hybrid beats flat MPI by about 13% (16x4 vs 64x1). With 64 ranks the 4x16 grid beat 8x8 (3274 vs 3126), 16x4 (2400) and 2x32 (2921).

Two nodes (32 ranks x 4 threads)

Grid P x Q (N 65280, NB 384)	1x32	2x16	4x8	4x4 (8 ranks x 8 thr)
GFLOPS	4560	5901	5222	4677

NB (N 67200, 2x16)	320	384	448
GFLOPS	5699	5873	5206

BCAST (N 67200)	1ring	1ringM	2ring	2ringM	Long	LongM
GFLOPS	5859	5923	5727	5796	3683	4227

DEPTH (N 67200)	0	1	2
GFLOPS	5673	5913	5880

Wide, flat grids win (P small, Q large) up to 2x16, where 1x32 falls off. NB = 384 is the sweet spot at both scales. The default 1ring-modified broadcast and look-ahead depth 1 were already the best; the long (bandwidth-reducing) broadcasts lose 29-38%. I did not sweep PFACT/RFACT/NBMIN/SWAP or the BLIS thread-partitioning variables; I expected under 2% there and stopped.

4. Final numbers

Run	Config	N	Time (s)	GFLOPS	% Rpeak
Best 1 node (tuning, 16x4, 4x4)	1 node	46,080	17.6	3702.8	50.9
Full size, in the repo (make run)	2 nodes, 2x16	231,552	1074.7	7701.6	53.0
Full size, clean-room test 1	2 nodes, 2x16	231,552	996.1	8309.3	57.1
Full size, clean-room test 2	**2 nodes, 2x16**	**231,552**	**968.1**	**8549.9**	**58.8**

N = 231,552 is 80% of the 2 x 250 GiB of RAM, rounded down to a multiple of NB. Efficiency rises with N because communication takes a smaller share. The 11% gap between identical full-size runs is not explained: I did not record which node pairs they used, and other users share the network. I suspect contention or node differences but have not tested it.

5. Reproducibility

`make` builds BLIS, Open MPI and HPL from the source tarballs shipped in `dist/` (no internet needed; I ran it with a dead HTTP proxy to prove that) and took about 3 minutes from a clean unpack. `make run` submits with `sbatch --wait`, so it blocks until HPL finishes and then prints the result table. The submitted `config/HPL.dat` is the one used. The wall time of `make run` in my last test was 49 minutes, of which 16 minutes were HPL and the rest queue wait.

6. What I learned

- Check the scheduler's limits early. A QOS cap decided the whole shape of the run, and I only found it because a job sat pending with an unfamiliar reason code.
- Look at what the job is doing, not only whether it is running. Resident memory and CPU time per job separated a hung start-up (10 MB, spinning) from real work (tens of GB, growing CPU time) in under a minute.
- Two jobs of mine overlapped once and one of them silently ran the other's `HPL.dat` (a 4x4 grid on 32 ranks, leaving half of them idle). The giveaway was memory: the whole matrix sat on one node. After that I ran one sweep at a time.
- On a 10 GbE link, grid shape and hybrid layout mattered more than any HPL algorithm knob. Large N also matters most: the same configuration went from 5.9 to 8.5 TFLOPS between N = 65k and N = 231k.